

Project Specification: Autotune in RTL!

1 Global Project Context

To shift the pitch of an audio signal without altering its tempo, we cannot simply speed up or slow down playback in the time domain. We must manipulate the audio in the frequency domain using a **Phase Vocoder** architecture.

The global pipeline for processing a complete audio file consists of the following steps:

1. **Ingestion:** Read floating-point audio samples from a file.
2. **Windowing:** Extract N -sized overlapping frames (e.g., 75% overlap) and apply a Hanning window to prevent spectral leakage.
3. **Transform:** Compute the Fast Fourier Transform (FFT).
4. **Conversion:** Convert complex FFT output to Magnitude and Phase.
5. **Pitch Detection:** Identify the fundamental frequency peak.
6. **Ratio Calculation:** Compare the peak against a standard tuning ROM table to find the shift factor S .
7. **Magnitude Resampling:** Scale the frequency bins to shift the pitch.
8. **Phase Unwrapping:** Maintain phase continuity across overlapping frames.
9. **Reconstruction:** Convert modified Magnitude and Phase back to complex numbers.
10. **Inverse Transform:** Compute the Inverse FFT (IFFT).
11. **Overlap-Add (OLA):** Sum overlapping frames.
12. **Export:** Write final samples to a new audio file.

2 Project Scope & Deliverables

This project focuses on the DSP datapath required for a single N -sized data window.

Core Objective (RTL Design)

Design, verify, and simulate a cycle-accurate Verilog datapath covering **Step 3 through Step 10**. The system will accept a single N -sized array of time-domain samples, compute the FFT, perform pitch shifting using CORDIC and interpolation, and output pitch-corrected samples.

Bonus Objective (Software Integration)

Develop a Python environment to:

- Extract windows from a `.wav` file
- Apply Hanning filter (Step 2)
- Perform Overlap-Add reconstruction (Step 11)
- Convert back to `.wav`

3 Phased Execution Strategy

3.1 Phase 1: Algorithmic Prototyping (Python)

Do not write Verilog yet.

- **Task:** Implement STFT, magnitude interpolation, phase unwrapping, and ISTFT using Python libraries such as `numpy`.
- **Goal:** Auto-tune a test `.wav` file. This acts as the **Golden Model**.

3.2 Phase 2: Digital System Design (Verilog RTL)

Map the algorithm into a fixed-point, pipelined hardware architecture.

4 Technical Architecture & Theory

4.1 Block A: Fast Fourier Transform (FFT)

The FFT converts time-domain signals into frequency-domain representation:

$$X[k] = a + jb$$

Each bin k represents a frequency component.

4.2 Block B: CORDIC Math Engine

We convert complex numbers into magnitude and phase:

$$\text{Magnitude} = \sqrt{a^2 + b^2}$$

$$\text{Phase} = \arctan\left(\frac{b}{a}\right)$$

CORDIC Solution:

- **Rectangular to Polar:** Outputs magnitude and phase
- **Polar to Rectangular:** Reconstructs $a + jb$

4.3 Block C: Pitch Shifting (Magnitude Interpolation)

To shift pitch by factor S :

$$\text{Source_Index} = \frac{\text{Target_Index}}{S}$$

If $\text{Source_Index} = 10.75$:

$$F = 0.75$$

$$\text{Mag}_{\text{New}} = (\text{Mag}_{10} \cdot (1 - F)) + (\text{Mag}_{11} \cdot F)$$

4.4 Block D: Phase Integrity (Phase Unwrapping)

Steps:

1. Compute phase deviation
2. Wrap to range:

$$[-\pi, \pi]$$

3. Accumulate corrected phase

This ensures continuity across frames.

4.5 Block E: Inverse Transform (IFFT)

The modified complex values are transformed back:

$$a + jb \rightarrow \text{time-domain samples}$$

Resulting in an N -sized array ready for Overlap-Add reconstruction.